

Requiero me ayudes con el siguiente ejercicio, detalla la demostración y define todo:

Demostrar que SAT es reducible en tiempo polinomial a CNF-SAT y a 3-SAT

Ha pensado durante 34s >

Vamos a demostrar formalmente

$$\text{SAT} \leq_p \text{CNF-SAT}$$

y

$$\text{SAT} \leq_p \text{3-SAT} .$$

La segunda demostración la obtendremos mediante

$$\text{SAT} \leq_p \text{CNF-SAT} \leq_p \text{3-SAT} .$$

La idea central será transformar fórmulas sin cambiar si son **satisfacibles**, usando variables auxiliares para evitar un crecimiento exponencial.

1. Definiciones necesarias

Una **variable booleana** es una variable que puede tomar valores

$$x \in \{0, 1\},$$

donde normalmente $1 = \text{verdadero}$ y $0 = \text{falso}$.

Un **literal** es una variable o su negación:

$$x, \quad \neg x.$$

Una **cláusula** es una disyunción de literales. Por ejemplo,

$$C = (x_1 \vee \neg x_2 \vee x_5).$$

Una fórmula está en **forma normal conjuntiva** o **CNF** (*Conjunctive Normal Form*) si es una conjunción de cláusulas:

$$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee \neg x_5).$$

Es decir, una CNF tiene la estructura

$$C_1 \wedge C_2 \wedge \dots \wedge C_m.$$

Por su parte, una fórmula está en **3-CNF** si cada cláusula contiene exactamente tres literales, o a lo sumo tres dependiendo de la definición utilizada. Por ejemplo,

$$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_4 \vee x_5).$$

Finalmente:

$$SAT = \{\varphi : \varphi \text{ es una fórmula booleana satisfacible}\}.$$

$$CNF-SAT = \{\varphi : \varphi \text{ es una CNF satisfacible}\}.$$

$$3-SAT = \{\varphi : \varphi \text{ es una 3-CNF satisfacible}\}.$$

Una fórmula es **satisfacible** si existe al menos una asignación de verdad que haga verdadera toda la fórmula.

2. ¿Qué significa una reducción polinomial?

Decir

$$A \leq_p B$$

significa que existe una función f , computable en tiempo polinomial, tal que

$$x \in A \iff f(x) \in B.$$

Para nuestro primer caso necesitamos construir una transformación f tal que

$$\varphi \in SAT \iff f(\varphi) \in CNF-SAT.$$

No es necesario que las dos fórmulas sean lógicamente equivalentes sobre exactamente

las mismas variables. Basta con que sean **equisatisfacibles**:

$$\varphi \text{ satisfacible} \iff f(\varphi) \text{ satisfacible.}$$

Esta diferencia será fundamental.

3. Por qué no basta con convertir directamente a CNF

Podríamos intentar usar identidades como

$$A \vee (B \wedge C) \equiv (A \vee B) \wedge (A \vee C).$$

El problema es que aplicar distributividad repetidamente puede producir un número exponencial de cláusulas.

Por ejemplo, una fórmula como

$$(x_1 \wedge y_1) \vee (x_2 \wedge y_2) \vee \cdots \vee (x_n \wedge y_n)$$

puede producir una CNF de tamaño exponencial al distribuir directamente.

Eso no sirve para demostrar una reducción polinomial.

La solución es la **transformación de Tseitin**.

4. Primera reducción: $\text{SAT} \leq_p \text{CNF-SAT}$

Supongamos que tenemos una fórmula arbitraria

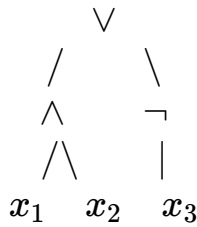
$$\varphi.$$

Podemos verla como un árbol formado por operaciones booleanas.

Consideremos, por ejemplo,

$$\varphi = (x_1 \wedge x_2) \vee \neg x_3.$$

Su estructura es aproximadamente



La transformación de Tseitin crea una variable nueva para representar el resultado de cada subfórmula.

Definimos

$$y_1 \leftrightarrow (x_1 \wedge x_2),$$

$$y_2 \leftrightarrow \neg x_3,$$

y

$$y_3 \leftrightarrow (y_1 \vee y_2).$$

Como y_3 representa toda la fórmula original, además exigimos

$$y_3 = 1.$$

Ahora necesitamos expresar esas equivalencias como CNF.

5. Codificación de cada operación

Operación AND

Supongamos

$$z \leftrightarrow (x \wedge y).$$

Esto significa que z tiene exactamente el mismo valor que $x \wedge y$.

La equivalencia puede escribirse como

$$(z \rightarrow x \wedge y) \wedge ((x \wedge y) \rightarrow z).$$

De aquí obtenemos

$$(\neg z \vee x) \wedge (\neg z \vee y) \wedge (z \vee \neg x \vee \neg y).$$

Por tanto,

$$z \leftrightarrow (x \wedge y)$$

se representa mediante

$$(\neg z \vee x) \wedge (\neg z \vee y) \wedge (z \vee \neg x \vee \neg y) .$$

Observa que solamente hemos creado **tres cláusulas**.

6. Operación OR

Para

$$z \leftrightarrow (x \vee y),$$

la CNF correspondiente es

$$(z \vee \neg x) \wedge (z \vee \neg y) \wedge (\neg z \vee x \vee y) .$$

De nuevo tenemos solamente tres cláusulas.

7. Operación NOT

Para

$$z \leftrightarrow \neg x,$$

obtenemos

$$(z \vee x) \wedge (\neg z \vee \neg x) .$$

Son solamente dos cláusulas.

La característica fundamental es que **cada operador de la fórmula original genera una cantidad constante de cláusulas**.

8. Aplicación al ejemplo

Consideremos nuevamente

$$\varphi = (x_1 \wedge x_2) \vee \neg x_3.$$

Introducimos

$$y_1 \leftrightarrow (x_1 \wedge x_2),$$

$$y_2 \leftrightarrow \neg x_3,$$

$$y_3 \leftrightarrow (y_1 \vee y_2).$$

Para y_1 tenemos

$$(\neg y_1 \vee x_1) \wedge (\neg y_1 \vee x_2) \wedge (y_1 \vee \neg x_1 \vee \neg x_2).$$

Para y_2 ,

$$(y_2 \vee x_3) \wedge (\neg y_2 \vee \neg x_3).$$

Para y_3 ,

$$(y_3 \vee \neg y_1) \wedge (y_3 \vee \neg y_2) \wedge (\neg y_3 \vee y_1 \vee y_2).$$

Finalmente agregamos

$$(y_3),$$

obligando a que la fórmula completa sea verdadera.

Por tanto, obtenemos una fórmula CNF

$$\psi = C_1 \wedge C_2 \wedge \cdots \wedge C_9.$$

9. ¿Por qué la transformación es correcta?

Necesitamos demostrar

$$\varphi \text{ satisfacible} \iff \psi \text{ satisfacible}.$$

Primero supongamos que φ es satisfacible. Entonces existe una asignación para las variables originales que hace

$$\varphi = 1.$$

Extendemos esa asignación a las variables nuevas haciendo que cada una almacene el resultado de su subfórmula:

$$y_1 = x_1 \wedge x_2,$$

$$y_2 = \neg x_3,$$

$$y_3 = y_1 \vee y_2.$$

Como φ es verdadera,

$$y_3 = 1.$$

Todas las equivalencias se cumplen y, por tanto, todas las cláusulas de ψ son verdaderas. Luego

ψ es satisfacible.

Ahora supongamos lo contrario: ψ es satisfacible.

Las cláusulas que construimos obligan a cumplir

$$y_1 = x_1 \wedge x_2,$$

$$y_2 = \neg x_3,$$

$$y_3 = y_1 \vee y_2.$$

Además tenemos la cláusula

$$(y_3),$$

por lo que necesariamente

$$y_3 = 1.$$

Así,

$$(x_1 \wedge x_2) \vee \neg x_3 = 1.$$

Por consiguiente,

φ es satisfacible.

Hemos demostrado

$$\varphi \in SAT \iff \psi \in CNF-SAT .$$

10. ¿Por qué la transformación es polinomial?

Supongamos que la fórmula original tiene tamaño

$$n = |\varphi|.$$

Por cada operador introducimos una variable auxiliar y una cantidad constante de cláusulas. Por ejemplo,

$$\wedge \rightarrow 3 \text{ cláusulas,}$$

$$\vee \rightarrow 3 \text{ cláusulas,}$$

$$\neg \rightarrow 2 \text{ cláusulas.}$$

Si hay $O(n)$ operadores, producimos solamente

$$O(n)$$

variables y

$$O(n)$$

cláusulas.

Por tanto,

$$|\psi| = O(|\varphi|).$$

La construcción se puede realizar recorriendo una sola vez el árbol sintáctico de φ , así que el tiempo también es

$$O(|\varphi|).$$

En particular, es polinomial. Hemos probado:

$$SAT \leq_p CNF-SAT .$$

11. Segunda parte: $CNF-SAT \leq_p 3-SAT$

Ahora partimos de una fórmula CNF

$$F = C_1 \wedge C_2 \wedge \cdots \wedge C_m.$$

El problema es que una cláusula puede tener muchos literales. Por ejemplo,

$$C = (x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5).$$

Necesitamos reemplazarla por cláusulas de exactamente tres literales.

12. Cláusulas con más de tres literales

Consideremos

$$C = (l_1 \vee l_2 \vee \cdots \vee l_k), \quad k > 3,$$

donde cada l_i es un literal.

Creamos variables nuevas

$$y_1, y_2, \dots, y_{k-3}.$$

Reemplazamos C por

$$(l_1 \vee l_2 \vee y_1)$$

$$\wedge (\neg y_1 \vee l_3 \vee y_2)$$

$$\wedge (\neg y_2 \vee l_4 \vee y_3)$$

$$\wedge \cdots \wedge$$

$$(\neg y_{k-4} \vee l_{k-2} \vee y_{k-3})$$

$$\wedge (\neg y_{k-3} \vee l_{k-1} \vee l_k).$$

Cada cláusula tiene exactamente tres literales.

13. Ejemplo con cinco literales

Tomemos

$$C = (x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5).$$

Aquí

$$k = 5.$$

Necesitamos

$$k - 3 = 2$$

variables auxiliares:

$$y_1, y_2.$$

Construimos

$$(x_1 \vee x_2 \vee y_1)$$

$$\wedge (\neg y_1 \vee x_3 \vee y_2)$$

$$\wedge (\neg y_2 \vee x_4 \vee x_5).$$

Esta fórmula pertenece a 3-CNF.

14. ¿Por qué esta transformación funciona?

Consideremos el caso general

$$C = (l_1 \vee \dots \vee l_k).$$

Queremos demostrar que

C es satisfacible

si y solo si las nuevas cláusulas pueden satisfacerse para alguna elección de las variables y_i .

La forma más interesante de verlo es considerar qué ocurriría si

$$l_1 = l_2 = \dots = l_k = 0.$$

La primera cláusula sería

$$(0 \vee 0 \vee y_1),$$

por lo que obliga a

$$y_1 = 1.$$

La segunda sería

$$(\neg y_1 \vee 0 \vee y_2).$$

Como $y_1 = 1$,

$$\neg y_1 = 0,$$

y entonces debe cumplirse

$$y_2 = 1.$$

El mismo razonamiento continúa:

$$y_1 = 1 \Rightarrow y_2 = 1 \Rightarrow y_3 = 1 \Rightarrow \dots \Rightarrow y_{k-3} = 1.$$

Pero la última cláusula sería

$$(\neg y_{k-3} \vee 0 \vee 0).$$

Como

$$y_{k-3} = 1,$$

tenemos

$$\neg y_{k-3} = 0.$$

Por tanto,

$$0 \vee 0 \vee 0 = 0.$$

Contradicción.

Así que es imposible satisfacer la transformación si todos los l_i son falsos.

Esto significa que necesariamente alguno debe ser verdadero:

$$l_1 \vee \dots \vee l_k = 1.$$

Es decir,

$$C = 1.$$

En la otra dirección, si algún l_i es verdadero, podemos elegir valores adecuados para los y_j para satisfacer toda la cadena.

Por ejemplo, podemos pensar en y_i como diciendo:

«Hasta este punto todavía no hemos encontrado un literal verdadero».

Formalmente podemos tomar

$$y_i = \neg(l_1 \vee l_2 \vee \dots \vee l_{i+1}).$$

Con esta asignación todas las cláusulas de la cadena quedan satisfechas siempre que la cláusula original C sea verdadera.

Por tanto,

$$C \text{ satisfacible} \iff C' \text{ satisfacible} .$$

15. ¿Qué hacemos con cláusulas de uno o dos literales?

Esto depende ligeramente de la definición de 3-SAT.

Si se define 3-SAT como cláusulas de **a lo sumo tres literales**, no tenemos que hacer nada.

Si exigimos **exactamente tres posiciones por cláusula**, podemos hacer transformaciones sencillas.

Para dos literales,

$$(l_1 \vee l_2)$$

introducimos una nueva variable y :

$$(l_1 \vee l_2 \vee y) \wedge (l_1 \vee l_2 \vee \neg y) .$$

¿Por qué funciona? Si

$$l_1 \vee l_2 = 1,$$

ambas cláusulas son verdaderas independientemente de y .

Pero si

$$l_1 = l_2 = 0,$$

obtenemos

$$y \wedge \neg y,$$

que es imposible.

Por tanto,

$$(l_1 \vee l_2)$$

y la nueva fórmula son equisatisfacibles.

Para una cláusula unitaria

$$(l_1),$$

podemos introducir dos nuevas variables y, z :

$$(l_1 \vee y \vee z)$$

$$\wedge (l_1 \vee y \vee \neg z)$$

$$\wedge (l_1 \vee \neg y \vee z)$$

$$\wedge (l_1 \vee \neg y \vee \neg z).$$

Si $l_1 = 1$, las cuatro cláusulas son verdaderas.

Si $l_1 = 0$, tendríamos simultáneamente

$$y \vee z, \quad y \vee \neg z, \quad \neg y \vee z, \quad \neg y \vee \neg z,$$

lo cual es imposible.

Así conservamos satisfacibilidad.

En algunas definiciones se permiten literales repetidos, caso en el cual simplemente podríamos escribir

$$(l_1 \vee l_1 \vee l_1),$$

pero la construcción anterior evita depender de esa convención.

16. Complejidad de CNF-SAT $\rightarrow$ 3-SAT

Sea una cláusula con $k > 3$ literales:

$$(l_1 \vee \dots \vee l_k).$$

Creamos

$$k - 3$$

variables nuevas y

$$k - 2$$

cláusulas.

Por tanto, el trabajo realizado es

$$O(k).$$

Si toda la fórmula tiene tamaño

$$n,$$

la suma de los tamaños de todas las cláusulas es $O(n)$, así que la nueva fórmula también tiene tamaño

$$O(n).$$

La transformación completa puede construirse en tiempo

$$O(n).$$

En consecuencia,

$$CNF-SAT \leq_p 3-SAT .$$

17. Finalmente: $SAT \leq_p 3-SAT$

Ya demostramos

$$SAT \leq_p CNF-SAT$$

y

$$CNF-SAT \leq_p 3-SAT.$$

Las reducciones polinomiales son transitivas. Es decir,

$$A \leq_p B$$

y

$$B \leq_p C$$

implican

$$A \leq_p C.$$

¿Por qué? Si tenemos

$$f : A \rightarrow B$$

y

$$g : B \rightarrow C,$$

simplemente aplicamos

$$h(x) = g(f(x)).$$

Como f se calcula en tiempo polinomial y g también, su composición sigue siendo polinomial.

Entonces

$$SAT \leq_p CNF-SAT \leq_p 3-SAT,$$

y por transitividad:

$$SAT \leq_p 3-SAT .$$

18. La demostración completa, vista conceptualmente

La cadena que acabamos de construir es

fórmula booleana arbitraria

$\Downarrow$ Tseitin

CNF

$\Downarrow$ dividir cláusulas largas

3-CNF .

Y en ambos pasos preservamos exactamente lo que nos interesa:

$$\text{satisfacible} \iff \text{satisfacible} .$$

Además, el tamaño solamente aumenta linealmente:

$$|\varphi| \longrightarrow O(|\varphi|) \longrightarrow O(|\varphi|).$$

Por tanto, las dos transformaciones son computables en tiempo polinomial y concluimos formalmente:

$$SAT \leq_p CNF-SAT$$

y

$$\text{SAT} \leq_p \text{3-SAT} .$$

Un punto especialmente importante para complejidad computacional es que aquí usamos **equisatisfacibilidad, no equivalencia lógica estricta**. Las nuevas fórmulas contienen variables que ni siquiera existían en la fórmula original; lo único necesario para la reducción es que exista una asignación satisfactoria en una fórmula **si y solo si** existe una en la otra.



Ramificado desde [**Demostrar Reducciones SAT CNF 3SAT**](#)

ChatGPT puede cometer errores. Considera verificar la información importante.